

CPS2010 Security by Design Assignment

Dave Galea

May 2026

Contents

0.1	Note about artefact reference throughout the report:	2
1	Task 1: Reliable and Robust Application Platforms	2
1.1	Task 1(a)	2
1.2	Task 1(b)	2
1.3	Task 1(c)	4
2	Task 2: Security Operations	5
2.1	Task 2(a)	5
2.2	Task 2(b)	6
2.3	Task 2(c)	7
2.3.1	API Services(fnd-ro & fnd-rw)	8
2.3.2	CNPG (fnd-db-1/2/3)	9
3	Task 3: Access Control and Resilience	10
3.1	Task 3(a)	10
3.2	Task 3(b)	10
4	Generative AI Usage	10
5	Artefact Index	10

0.1 Note about artefact reference throughout the report:

Unless stated otherwise, each referenced path for an artefact is relative to the folder of the relevant subtask. For example, when `yamls/cnpg-cluster.yaml` is referenced in Task 1a, it refers to `task1a/yamls/cnpg-cluster.yaml` in the submitted Google Drive folder.

1 Task 1: Reliable and Robust Application Platforms

1.1 Task 1(a)

CloudNative Operator deployment

The original single PostgreSQL setup was replaced with a CloudNativePG-managed PostgreSQL cluster. The CloudNativePG Helm repository was added, and the CNPG operator was installed in the `cnpg-system` namespace using Helm, using appropriate documentation [?] as reference.

Cluster bootstrap

The PostgreSQL cluster was created using `task1a/yamls/cnpg-cluster.yaml`. This manifest created the Kubernetes secret `app-secret` and bootstrapped the `fnd-db` cluster with the `ordersdb` database owned by `clusteruser`. Reference for how to perform this step was obtained from `cloudnative-pg.io` documentation [?].

Read-write/read-only split

CloudNativePG exposed the required split through separate Kubernetes services. The `fnd-db-rw` service targets the current primary instance and is used for read-write traffic. The `fnd-db-ro` service targets replica instances and is used for read-only traffic.

The cluster was configured with `instances:3`. For the read-write/read-only split, 2 instances would have been sufficient, since this would provide one primary instance and one replica. Although the additional replica was not strictly required for the task, it improved the final setup by increasing the capacity available for read-only traffic.

Resource verification

Correct Kubernetes resource creation and pod deployment was confirmed using `kubectl`. All commands used can be found in the `task1a readme.txt`

1.2 Task 1(b)

Preparing the Repository

After splitting the codebase into respective versions, one for read-write, and one for read-only, both versions were prepared to perform their respective tasks. Naturally most configuration was applied to the read-only version of the app, as it is the version which required inhibition of previously available features.

Changes made to the read only codebase include:

- Applying of a `RepositoryRestConfigurer` to block non-GET requests (file at `app-ro/src/main/java/com/example/foodndeliv/config/SpringDataRestCustomization.java`).

Reference code was used from 2nd set of notes: Reliable Application Platforms.pdf, from pages 124-125.

- Modifying controllers and services in the source code to block any mappings (such as put and post) exposed which may allow non-GET requests, as well as comment out any keycloak-dependent code.
- Disabling of keycloak config files, as they were not needed for this task

Changes made to both repositories:

- Modifying the docker files accordingly
- Adding an exception handler for HTTP methods, which shows specific errors for when a method is not supported (eg, POST on a read-only service).
- Modifying application.properties. Most importantly `spring.jpa.hibernate.ddl-auto=...`. This was set as `=update` for the first building of the RW app, and then as `=none` for the second rebuilding, for reasons explained in the **Schema Setup** section below.

Connecting the services to the databases

The `fnd-rw` deployment sets `DB_HOST=fnd-db-rw`, while the `fnd-ro` deployment sets `DB_HOST=fnd-db-ro`.

The deployments and services were created using `yamls/rw-deployment.yaml`, `yamls/ro-deployment.yaml`, `yamls/rw-service.yaml`, and `yamls/ro-service.yaml`. The resulting services were `fnd-rw` and `fnd-ro`. Both deployments use the credentials defined in `yamls/db-secret.yaml`. The yaml file configuration was inspired from the [kubernetes documentation](#)[?] [?] [?]

Deployment

Both applications were built as separate Maven projects and Docker images. The read-write image was pushed as `davegalea/foodndeliv-rw:task1b`, and the read-only image was pushed as `davegalea/foodndeliv-ro:task1b`.

Schema setup

For the first read-write deployment, `spring.jpa.hibernate.ddl-auto=update` was used so that the database schema could be create. After this, the read-write application was rebuilt with `spring.jpa.hibernate.ddl-auto=none`. The read-only version also used `ddl-auto=none`, since it should not create or modify the schema.

Testing and verification

The services were exposed locally using port forwarding:

```
kubectl port-forward svc/fnd-ro 8080:8080 (for read only)
kubectl port-forward svc/fnd-rw 8081:8080 (for read-write)
```

The read-only API accepted GET requests and rejected non-GET requests, while the read-write API retained the original behaviour, rejecting only DELETE requests as specified to do so in the untouched source code. Various tests were performed, all listed in **postman/tests.txt**.

The deployment was verified using `kubectl get pods`, `kubectl describe deployment fnd-rw`, `kubectl describe deployment fnd-ro`, and the pod logs for both deployments.

1.3 Task 1(c)

i) Least Privilege

Least Privilege is the principle where only the minimum amount of permissions necessary to perform a task are allocated to a program and/or its user.[?]

This setup applies Least Privilege by separating the FoodNDeliv API into two deployments: **fnd-rw** and **fnd-ro**. The **fnd-rw** deployment is used for operations that require database writes and is connected to the CNPG read-write service, while the **fnd-ro** deployment is used only for read operations and is connected to the CNPG read-only service. The cited definition is primarily exemplified through the read-only API version. In addition to using the CNPG read-only service, it is restricted to only accept GET requests, meaning that the service has no other permissions besides those needed for its role.

ii) Handling traffic spikes

This setup handles traffic spikes mainly by separating read-only and read-write traffic. This means that a surge in read-heavy traffic can be directed to the read-only API instead of placing all traffic on the read-write API.

The CNPG cluster was deployed with **instances:3**, providing one primary PostgreSQL instance and two replica instances. This further supports read-heavy workloads because read-only database traffic can be served through the CNPG read-only service, which routes to replica instances, rather than forcing all database traffic through the read-write primary instance.

iii) Availability during failures

Availability is improved because the CNPG database is deployed with multiple PostgreSQL instances rather than relying on a single database pod. The current system has one primary instance and two replica instances, so the system is not dependent on only one PostgreSQL pod remaining healthy.

If the primary instance fails, CNPG can promote one of the existing replicas to become the new primary, allowing read-write database access to continue after failover.

iv) Data Loss Prevention

Data loss prevention is achieved through both replication and restriction of DELETE operations.

Regarding replication, the CNPG cluster is deployed with one primary instance and two replica instances, so database state is not dependent on a single PostgreSQL pod. Data written to the primary is thus replicated, so if the primary pod fails, one of the replicas can be promoted and service can continue with a copy of the database state.

Regarding restriction, the setup also restricts DELETE operations in both API versions. The read-only API, only accepts GET requests and therefore by nature cannot cause data loss via deletion. connects to the CNPG read-only service, **fnd-db-ro**. Therefore, it cannot be used to create, update, or delete records. The read-write API can modify data, but delete operations are also disabled due to `@RestResource(exported=false)` above the `deleteById` methods for all three Customer, Order, and Restaurant Repository files.

2 Task 2: Security Operations

2.1 Task 2(a)

The Baseline profile controls are those which adhere to the kubernetes baseline policy, which serves as a middle ground between an unrestricted and open privileged policy, and a restricted policy, which aims to enforce best practices for security-critical applications.[?]

The table below addresses the Linux-applicable Baseline controls, the isolation each control provides, and how each one limits what an attacker can do after a pod has been compromised (post-exploitation)

Table 1: Linux-targeting Baseline Profile controls and their post-exploitation impact.

Baseline control	Isolation provided	How it limits post-exploitation
Host Namespaces	Disallows sharing the host network, host process identifier (PID), and host inter-process communication (IPC) namespace.	If a pod is compromised, the attacker is restricted from using the hosts's network namespace, viewing information about host processes, or interfering with host inter-process communication.
Privileged Containers	This control disallows containers from running with <code>securityContext.privileged: true</code> . Privileged containers are purposely-open and entirely unrestricted.	This control is preventive at admission time, because it stops privileged pods from being deployed. It limits post-exploitation because if the pod is later compromised, the attacker does not inherit the lack of restriction of privileged containers.
Capabilities	This control restricts extending the set of allowed capabilities from a defined list. Allowed capabilities included <code>AUDIT_WRITE</code> , <code>CHOWN</code> , and others listed in the kubernetes documentation.	By disabling the ability to extend the list of capabilities, compromised pods cannot do anything besides those within the baseline policy.
HostPath Volumes	This control forbids <code>hostPath</code> volumes, which would otherwise allow a pod to mount files or directories from the host's filesystem.	If a compromised pod cannot mount host paths, the attacker cannot use the pod as a direct route to read or modify sensitive host files.
Host Ports	This control disallows or restricts the use of <code>hostPort</code> , preventing containers from binding directly to the host's ports.	An attacker with access to a compromised pod cannot easily expose a malicious service directly on the host's network.

Continued on next page

Baseline control	Isolation provided	How it limits post-exploitation
Host Probes / Lifecycle Hooks	This control disallows setting the <code>host</code> field in HTTP or TCP probes and lifecycle hooks.	If a pod is compromised, the attacker cannot rely on probe or lifecycle hook configuration to direct container checks or lifecycle actions toward arbitrary hosts not previously configured.
AppArmor	This control prevents disabling or overriding the AppArmor profile, a linux kernel security module which protects applications through strict security profiles [?].	Even if a pod is compromised, the AppArmor profile can still restrict what the container process is allowed to access or execute.
SELinux	This control restricts modification of Security-Enhanced (SE)Linux options.	A compromised pod cannot relieve itself of the strict policy that SELinux enforces, which is a mandatory access control system [?].
/proc Mount Type	<code>/proc</code> is a virtual files system that provides information about active processes [?]. This control requires the default <code>/proc</code> mount behaviour, and prevents it from being unmasked.	If a pod is compromised, the attacker has less visibility into host and kernel information because sensitive <code>/proc</code> paths remain masked.
Seccomp	This control prevents explicitly setting the Secure Computing Mode (seccomp) profile to Unconfined . Seccomp isolates processes by filtering and thus restricting which system calls they can invoke [?].	Keeping seccomp enabled reduces dangerous or unnecessary syscalls that can be called after pod compromise, making post-exploitation harder.
Sysctl	This control disallows unsafe sysctls which can otherwise change kernel parameters affecting all containers on a host. [?]	If a pod is compromised, the attacker cannot use unsafe sysctls to affect node-level settings, or other pods on the same node.

2.2 Task 2(b)

The Restricted Profile raises the level of isolation beyond the Baseline Profile by enforcing stricter container hardening rules on top of the Baseline policy. It has various additional controls: some new ones, and some which build upon the previous controls in baseline.

Volume types:

The volume type control builds upon the Baseline restriction on `hostPath` volumes. Baseline prevents pods from mounting arbitrary files or directories from the host filesystem through `hostPath`. Restricted goes further by limiting pods to a small set of approved Kubernetes-managed volume types, such as `configMap`, and others listed on the Kubernetes documentation[?]. This improves isolation by ensuring that storage access remains within controlled Kubernetes volume types.

Privilege escalation:

The privilege escalation control builds upon the Baseline restriction on privileged containers. Baseline is preventive at admission time because it stops containers from being launched with `securityContext.privileged: true`, which blocks the most unrestricted form of container execution before the pod is deployed. Restricted goes further by requiring `allowPrivilegeEscalation: false`, withholding from the pod's ability to gain that privilege later on in its lifetime. This improves isolation because if the pod is compromised, the attacker remains limited to the privileges of the compromised process rather than being able to escalate further inside the container.

Running as Non-root:

`runAsNonRoot` is the Restricted-Policy specific requirement that the container must not run as root, while `runAsUser` is the specific Linux user ID selected for the container process. Restricted policy uses both controls so that a pod must declare non-root execution and must not explicitly set the user ID 0, which is the root user. This raises isolation because a compromised process has fewer permissions inside the container than it would have if it were running as root.

Seccomp:

The seccomp control builds directly upon the Baseline seccomp rule. Seccomp limits the system calls that a container process can make to the Linux kernel. Baseline prevents a pod from explicitly setting seccomp to `Unconfined`, which blocks deliberately disabling system call filtering. However, Baseline may still allow the seccomp profile to be unset. Restricted is stricter because it requires the pod or container to explicitly use either `RuntimeDefault` or `Localhost`. Restricted requires an allowed seccomp profile to be declared. This improves isolation because if the container is compromised, the attacker has fewer system-calls available for post-exploitation.

Capabilities:

The capabilities control also builds directly upon the Baseline capabilities rule. Baseline restricts which additional Linux capabilities may be added to a container. Restricted, on the other hand is stricter because it requires containers to drop all capabilities and only allows `NET_BIND_SERVICE` to be added back; thus only allowing the capability of binding a server socket to a privileged port[?]

2.3 Task 2(c)

To measure how well the current setup, which includes both the API service and CNPG, comply with the two Pod Security Standards policy profiles, `kubectl` was used. The following pods were inspected individually:

- `fnd-ro-b67f56765-9wdfg` (API service)
- `fnd-rw-84559864d4-g9q24` (API service)
- `fnd-db-1` (CNPG)
- `fnd-db-2` (CNPG)
- `fnd-db-3` (CNPG)

The pod specifications were exported using: `kubectl get pod <pod-name> -o yaml`

The above can be seen in `task2c/kubectl-session.txt`

2.3.1 API Services(fnd-ro & fnd-rw)

Baseline profile.

- **Host namespaces: Compliant** as `spec.hostNetwork`, `spec.hostPID`, and `spec.hostIPC` are absent.
- **Privileged containers: Compliant** as `securityContext.privileged` is set to `false`.
- **Capabilities: Compliant** as `containers[*].securityContext.capabilities.add` is absent
- **HostPath volumes: Compliant** as `spec.volumes[*].hostPath` is absent.
- **Host ports: Compliant** `containers[*].ports[*].hostPort` is absent
- **Host probes / lifecycle hooks: Compliant** as their configuration is absent.
- **AppArmor: Compliant** as AppArmor configuration is absent.
- **SELinux: Compliant** as SELinux configuration is absent.
- **/proc mount type: Compliant** as `procMount` configuration is absent.
- **Seccomp: Compliant** as `seccompProfile` is absent and thus not set to `Unconfined`.
- **Sysctls: Compliant** as `sysctls` configuration is absent.

Restricted profile.

- **Everything from Baseline: Compliant.**
- **Volume types: Compliant** as volume type `projected:` is used.
- **Privilege escalation: Compliant** as Privilege escalation configuration is set to `false`
- **Running as non-root: Not Compliant** as `runAsNonRoot` configuration is absent and Restricted profile requires it to be set as `true`
- **Running as non-root user: Compliant** as `runAsUser` is absent, and for the Restricted profile, it must not be 0.
- **Seccomp: Not Compliant** as Restricted profile requires `seccompProfile.type` to be explicitly set to `RuntimeDefault` and it is absent.
- **Capabilities: Not Compliant** as Restricted profile requires `capabilities.drop` to include `ALL`, and `capabilities.drop` is absent.

Result. The API services are Baseline Profile compliant, however violate a few requirements necessary to be Restricted Profile Compliant.

2.3.2 CNPG (fnd-db-1/2/3)

Baseline profile.

- **Host namespaces: Compliant** as `spec.hostNetwork`, `spec.hostPID`, and `spec.hostIPC` are absent.
- **Privileged containers: Compliant** as `securityContext.privileged` is set to `false`.
- **Capabilities: Compliant** as `securityContext.capabilities.add` is absent
- **HostPath volumes: Compliant** as `spec.volumes[*].hostPath` is absent.
- **Host ports: Compliant** as `containers[*].ports[*].hostPort` is absent
- **Host probes / lifecycle hooks: Compliant** as probe configuration do not set a custom host, and lifecycle hooks are absent.
- **AppArmor: Compliant** as AppArmor configuration is absent.
- **SELinux: Compliant** as SELinux configuration is absent.
- **/proc mount type: Compliant** as `procMount` configuration is absent.
- **Seccomp: Compliant** as `seccompProfile` is set to `RunTimeDefault` and is thus not unconfined
- **Sysctls: Compliant** as `sysctls` configuration is absent.

Restricted profile.

- **Everything from Baseline: Compliant.**
- **Volume types: Compliant** as volume type `projected` is used.
- **Privilege escalation: Compliant** as Privilege escalation configuration is set to `false`.
- **Running as non-root: Compliant** as `runAsNonRoot` configuration is set as `true`.
- **Running as non-root user: Compliant** as `runAsUser` is set to 26, and for the Restricted profile, it must not be 0.
- **Seccomp: Compliant** as Restricted profile requires `seccompProfile.type` to be explicitly set to `RuntimeDefault`, and it is set as so.
- **Capabilities: Compliant** as Restricted profile requires `capabilities.drop` to include `ALL`, and it is set as so.

Result. The CNPG Pods comply with both Baseline and Restricted Profiles.

3 Task 3: Access Control and Resilience

3.1 Task 3(a)

3.2 Task 3(b)

4 Generative AI Usage

5 Artefact Index